

Vector Databases and Retrieval Systems for Large-Scale AI Applications

Abstract

The paradigm shift in artificial intelligence from structured, rule-based systems to representation learning has established high-dimensional vector embeddings as the universal substrate for unstructured data. As massive datasets of text, images, audio, and video are continuously encoded by deep neural networks, the necessity to store, index, and query these continuous representations has catalyzed the rapid evolution of Vector Database Management Systems (VDBMS). This survey provides an exhaustive, academic examination of the core algorithmic foundations, system-level architectures, and query processing frameworks that define modern vector retrieval systems. Traditional relational databases, optimized for exact matches and B-tree index traversals, are fundamentally ill-equipped to navigate the geometric complexities of high-dimensional spaces, where the "curse of dimensionality" degrades exact nearest neighbor search to linear time complexity. Consequently, Approximate Nearest Neighbor Search (ANNS) has emerged as the definitive operational standard. This manuscript systematically categorizes the algorithmic landscape into graph-based, quantization-based, hashing-based, tree-based, and learned indexing methodologies, detailing the theoretical mechanics and mathematical trade-offs inherent in each approach.

Furthermore, the survey explores the critical architectural transition from purely in-memory indexes to highly scalable memory-disk hybrid systems, analyzing frameworks such as DiskANN, SPANN, and HM-ANN, which leverage solid-state drives and heterogeneous memory (PMem) to support billion-scale corpora. Crucially, the analysis delves into the complexities of vector-scalar hybrid queries (filtered search), evaluating pre-filtering, post-filtering, and modern joint-execution frameworks—such as VBASE and SIEVE—that seamlessly unify relational structured predicates with semantic proximity. Finally, the report investigates distributed cloud-native infrastructures, hardware acceleration via Graphics Processing Units (GPUs) and Field-Programmable Gate Arrays (FPGAs), and the emerging challenges of dynamic real-time graph updates and multimodal benchmarking. This manuscript serves as a definitive, structured resource for understanding the technical depth and future trajectory of vector retrieval systems in the era of generative artificial intelligence and Retrieval-Augmented Generation (RAG).

Introduction

The proliferation of deep learning architectures, particularly Transformer-based Large Language Models (LLMs) and Vision Transformers, has fundamentally altered the paradigm of data processing and information retrieval¹. Unstructured data modalities—comprising text, images, audio, and video—are now routinely encoded into high-dimensional vector embeddings. In these continuous semantic spaces, the contextual and semantic similarity between two distinct real-world entities is quantified by their geometric proximity, typically

measured via spatial metrics such as Euclidean distance (L_2), inner product, or cosine similarity³. The core computational challenge in this environment is the k -Nearest Neighbor (k -NN) search⁶.

Formally, given a dataset of high-dimensional vectors $D \subset \mathbb{R}^d$, a query vector $q \in \mathbb{R}^d$, and a distance function $dist(\cdot, \cdot)$, the exact k -NN search seeks to retrieve a subset $R \subseteq D$ where $|R| = k$, such that for every $x \in R$ and $y \in D \setminus R$, $dist(q, x) \leq dist(q, y)$ ⁹. While the exact k -NN search is trivially solvable via exhaustive

linear scanning in $O(n \cdot d)$ time, the phenomenon known as the "curse of dimensionality" renders exact retrieval computationally intractable at web-scale. In high-dimensional spaces, the contrast between the nearest and farthest points diminishes, causing deterministic spatial partitioning structures like KD-trees to degrade to the performance of a brute-force scan¹¹. To circumvent these computational bottlenecks, the field has universally adopted Approximate Nearest Neighbor Search (ANNS). ANNS relaxes the strict exactness guarantee in exchange for orders-of-magnitude improvements in query latency and throughput, aiming to retrieve an

approximation of the true top- k set⁶. The accuracy of an ANNS algorithm is strictly evaluated using the metric of *recall*, defined as the intersection of the retrieved approximate set with the true exact set, divided by k . Over the past decade, isolated ANNS libraries such as FAISS have demonstrated exceptional capabilities in memory-bound environments¹⁶.

However, integrating these algorithms into production artificial intelligence applications—such as Retrieval-Augmented Generation (RAG), semantic recommendation systems, and autonomous agent memory—requires substantially more than algorithmic efficiency². A modern Vector Database Management System (VDBMS) must extend raw ANNS capabilities with traditional database features, including persistent storage, Atomicity, Consistency, Isolation, and Durability (ACID) guarantees, Multi-Version Concurrency Control (MVCC), distributed sharding, and real-time streaming updates¹⁸. Furthermore, real-world workloads rarely require pure semantic search; they predominantly consist of vector-scalar hybrid queries, where a semantic search must be constrained by rigid structured predicates (e.g., retrieving the most semantically similar documents published within a specific chronological window)²⁰. This survey systematically unpacks these layers, progressing from the mathematical foundations of vector indices to the architectural designs of distributed, hardware-accelerated vector databases.

Methods

The architectural and algorithmic space of vector databases can be organized into distinct methodological families. This section deconstructs the indexing algorithms that power

similarity search, the storage paradigms that allow them to scale out-of-core, the frameworks used to process complex filtered queries, and the systems engineering required to accelerate and distribute these workloads.

Approximate Nearest Neighbor Search Algorithms

The retrieval engine of any vector database relies on an underlying ANNS index. These indexes determine the fundamental trade-offs between memory footprint, query latency, and recall accuracy. The primary paradigms include graph-based, quantization-based, hashing-based, tree-based, and learned indexing methodologies.

Algorithm Family	Representative Methods	Core Mechanism	Computational Trade-offs
Graph-Based	HNSW, NSG, Vamana	Navigable proximity graphs with greedy best-first routing.	Highest recall and lowest search latency; consumes massive memory and incurs long index build times.
Quantization	PQ, OPQ, ScaNN	Compresses vectors into sub-space centroids to estimate distances.	Highly memory-efficient, enables fast distance computation, but suffers from inherent quantization accuracy loss.
Hashing	LSH, Deep Hashing	Maps geometrically similar vectors to identical binary hash buckets.	Sub-linear search time with theoretical guarantees; performance heavily relies on hash function distribution.
Tree-Based	KD-Tree, VP-Tree, M-Tree	Hierarchical spatial partitioning via hyperplanes or	Optimal for low dimensions (

		hyperspheres.	$d < 20$); degrades severely to linear scanning in high-dimensional spaces.
Learned Indexing	RMI, LIDER, SHG	Neural networks predict the storage location or skipping levels of vectors.	Fast inference and minimal storage overhead; highly challenging to update dynamically under streaming writes.

Graph-Based Approaches

Graph-based ANNS has emerged as the dominant paradigm in modern vector databases due to its superior recall-latency profile²³. These methods construct a proximity graph where each vector acts as a node, and edges connect vertices that are geometrically proximate in the high-dimensional space. Query execution relies on a greedy best-first search strategy: starting from a designated entry node, the algorithm iteratively evaluates the neighbors of the current

node and transitions to the neighbor that minimizes the distance to the query vector q , terminating when a local minimum is reached²⁵.

The **Hierarchical Navigable Small World (HNSW)** graph is the most universally deployed algorithm in both dedicated and extended vector databases³. HNSW constructs a multi-layer structure where edges are segregated by length. The highest layers contain only sparse, long-range links, allowing the search to rapidly traverse the global vector space (the "zoom-in" phase) in logarithmic time complexity. As the search descends to lower, denser layers, short-range edges facilitate fine-grained local refinement. During index construction, the

maximum layer l for an inserted element is determined stochastically using an exponentially decaying probability distribution, ensuring that the hierarchical topology naturally mirrors the properties of a probabilistic skip list³.

To address the exceptionally high memory footprint of HNSW, the **Navigating Spreading-out Graph (NSG)** was developed as an approximation of the Monotonic Relative Neighborhood Graph (MRNG)⁴. NSG flattens the index into a single layer and drastically reduces the average out-degree of the graph. It achieves this by applying a strict edge-selection heuristic based on

the triangle inequality: an edge between node u and neighbor v is pruned if there exists a closer neighbor w that already connects them. This geometric pruning minimizes redundant

search paths, guarantees monotonic convergence towards the query, and significantly reduces the overall index size⁴.

The **Vamana** algorithm, the foundational graph structure behind the DiskANN system, introduces the RobustPrune mechanism to balance short-range exploitative edges with long-range exploratory edges on a single, disk-friendly layer²⁸. Given a candidate set of

neighbors for a node p , RobustPrune iteratively connects p to the closest candidate p^* , and then aggressively removes any other candidate p' from the pool if the distance from p^* to p' is less than $\alpha \cdot \text{dist}(p, p')$, where $\alpha \geq 1$ is a tunable density parameter. This mechanism ensures that the graph remains highly connected across diverse clusters while strictly bounding the maximum node degree²⁸.

Quantization-Based Approaches

While graph algorithms optimize traversal pathing, quantization algorithms optimize distance computation latency and raw memory consumption. **Product Quantization (PQ)** is the seminal method in this space²⁹. PQ addresses the memory bottleneck of storing high-dimensional

float32 vectors by decomposing the original D -dimensional vector space into m mutually orthogonal sub-spaces, each of dimension D/m . A k -means clustering algorithm is applied independently to each sub-space to generate k^* centroids (typically $k^* = 256$, allowing each sub-vector to be represented by a single 8-bit integer). Consequently, a dense

1024-dimensional float32 vector (4096 bytes) can be compressed into just m bytes²⁹. During retrieval, Asymmetric Distance Computation (ADC) is employed: the uncompressed query

vector q is compared against the precomputed centroid distances of the compressed database vectors via fast CPU lookup tables, enabling exhaustive searches over millions of vectors in milliseconds³².

A critical advancement in this domain is **Anisotropic Vector Quantization**, introduced by Google's ScaNN library³⁴. Traditional PQ algorithms minimize the Mean Squared Error (MSE),

defined as the reconstruction error $\mathbb{E}[\|x - \tilde{x}\|^2]$ between the original vector x and its

quantized representation $\tilde{x}$. However, in Maximum Inner Product Search (MIPS)—the standard metric for evaluating dot-product attention in recommendation systems and neural network

classifiers—minimizing global MSE is structurally suboptimal³⁴. The inner product $\langle q, x \rangle$ is

disproportionately affected by quantization errors that lie parallel to the vector x rather than orthogonal to it. ScaNN introduces an anisotropic loss function that weights parallel residual errors higher than orthogonal errors. By intentionally trading off increased overall quantization error in exchange for superior angular preservation, ScaNN achieves state-of-the-art MIPS

accuracy, allowing the quantized inner products to preserve the exact ranking of the unquantized original vectors³⁴.

Hashing and Tree-Based Approaches

Locality-Sensitive Hashing (LSH) utilizes families of random hash functions specifically designed such that the probability of collision for two vectors is strictly proportional to their geometric similarity⁶. In the Euclidean space, LSH projects vectors onto random hyperplanes and segments them into discrete bins. By utilizing multiple hash tables, LSH provides sub-linear retrieval times with rigorous mathematical guarantees on approximation quality¹⁴. However, standard LSH requires substantial memory to store the numerous hash tables required to maintain high recall. Deep hashing methods have recently gained traction, utilizing convolutional and dense neural networks to map high-dimensional representations directly into compact, topology-preserving binary codes, thereby enabling ultra-fast Hamming distance calculations via hardware-level bitwise XOR operations³⁸.

Tree-based methods, including KD-Trees, Vantage Point Trees (VP-Trees), and M-Trees, rely on hierarchical space partitioning¹¹. KD-Trees recursively bisect the coordinate space along the axis of maximal variance. While mathematically exact and highly efficient in low-dimensional spaces ($d < 20$), their performance rapidly deteriorates in high dimensions. Because the volume of the search hypersphere intersects almost all high-dimensional partitioning hyperplanes, the search algorithm is forced to backtrack and explore the majority of the tree nodes¹¹. Modern implementations mitigate this degradation by deploying forests of randomized KD-trees and strictly limiting the backtracking queue, transforming the exact search into an approximate one¹³.

Learned Indexing for Vector Spaces

Taking inspiration from the success of learned indexes for structured relational data, researchers have developed **Learned Indexes** for high-dimensional vector retrieval⁴². The Recursive Model Index (RMI) replaces classical B-tree branching logic with a hierarchy of neural networks or linear regression models that predict the physical position of a record based on its key. For continuous vector spaces, models like **LIDER** apply dimensionality reduction (e.g., SK-LSH) to map dense embeddings into one-dimensional sortable hash keys, which are then modeled by RMIs to predict the exact memory offset for the nearest neighbor scan⁴². Furthermore, predictive models like **Shortcut-enabled Hierarchical Graphs (SHG)** optimize graph search by training a predictive cutoff table that safely skips redundant upper-layer graph traversals, estimating the optimal entry layer for each specific query to minimize distance computations⁴⁴. Similar learned constraints are utilized in **LotusFilter**, which applies a learned cutoff table as a post-processing module to maximize the diversity of retrieved results by dynamically pruning geometrically redundant candidates⁴⁵.

Vector Database Architectures and Storage Management

As vector datasets expand from millions to billions of vectors, the assumption that the entire index and vector payload can reside in dynamic random-access memory (DRAM) is no longer

economically viable. For a dataset of one billion 128-dimensional float32 vectors, the raw data alone requires over 500 gigabytes of memory, and graph indexes like HNSW inflate this requirement to well over a terabyte per node. Consequently, VDBMS architectures have evolved out-of-core, memory-disk hybrid, and fully distributed cloud-native designs⁴⁶.

Memory-Disk Hybrid Systems

DiskANN represents a watershed architectural shift by proving that commodity Solid-State Drives (NVMe SSDs) can serve billion-scale graph queries with sub-5 millisecond latency²⁸. DiskANN leverages the Vamana graph algorithm. During index construction, the complete graph topology and the full-precision dense vectors are persisted sequentially on the SSD, ensuring that each node and its corresponding adjacency list occupy a contiguous block (typically aligned to 4KB sector sizes) to maximize sequential read bandwidth²⁸. In DRAM, DiskANN caches only a highly compressed PQ representation of the vectors. Query routing is performed entirely in memory using the compressed PQ vectors; when the search identifies the candidate neighborhood, the system issues parallel asynchronous I/O requests to fetch the full-precision vectors from the SSD for exact distance re-ranking. By decoupling the routing structure from the payload, DiskANN achieves the high recall of HNSW while drastically slashing memory infrastructure costs²⁸.

SPANN (Highly-efficient Billion-scale Approximate Nearest Neighbor Search) takes an alternative approach by modernizing the Inverted File (IVF) index⁴³. SPANN utilizes a hierarchical balanced clustering algorithm to partition the billion-scale dataset into thousands of distinct clusters (posting lists). The centroids of these clusters are kept in DRAM, while the massive posting lists containing the actual vector data are relegated to the SSD. During a query, SPANN dynamically evaluates the query vector against the in-memory centroids to determine which posting lists to fetch from disk. To mitigate the boundary problem—where a query vector falls on the geometric edge of a cluster, necessitating expensive multi-cluster disk reads—SPANN physically replicates boundary vectors across adjacent posting lists (a technique termed closure augmentation). This redundancy ensures that high recall can be achieved with a minimal number of random disk accesses⁴³.

HM-ANN advances storage design by exploiting Heterogeneous Memory (HM), specifically architectures combining standard DRAM with persistent memory (PMem) technologies¹⁸. PMem offers significantly higher capacity than DRAM but with asymmetric read/write latencies. HM-ANN maps the hierarchical nature of HNSW directly onto this hardware topology. The upper, sparse layers of the graph—which are frequently accessed during the initial zoom-in phase of every query—are pinned to the fast DRAM. The dense, bottom-most layer (Layer 0), which contains the complete dataset and consumes the vast majority of memory, is allocated entirely to PMem¹⁸. HM-ANN employs a specialized bottom-up promotion strategy, selectively promoting nodes with the highest connectivity degree from the PMem layer into the DRAM layers, optimizing cross-memory bandwidth and achieving performance that tightly matches pure in-memory systems¹⁸.

Distributed and Cloud-Native VDBMS Architectures

To support multi-tenant scale-out, systems like **Milvus** and its evolutionary successor **Manu**

are architected entirely around cloud-native, microservice-based principles¹⁸. Manu utilizes a strictly disaggregated architecture consisting of four layers: the access layer (stateless endpoint proxies), the coordinator layer (global metadata management), the worker layer (executing parallel ANNS queries), and the storage layer (persistent object storage like Amazon S3)¹⁹.

The cornerstone of the Manu architecture is its Log-Structured backbone. Write operations are appended to a distributed Write-Ahead Log (WAL) functioning as an immutable publish/subscribe stream¹⁹. Query nodes subscribe to this log, fetching delta updates directly into memory to provide near-real-time search availability. Because vector similarity applications rarely require strict global ACID transactions, Manu prioritizes tunable consistency, allowing clients to specify a tolerable time-tick lag. This Multi-Version Concurrency Control (MVCC) approach maximizes read throughput by removing the bottleneck of distributed lock mechanisms¹⁹.

To handle horizontal distribution, vector databases employ various sharding strategies⁵⁰. Hash-based sharding utilizes consistent hashing algorithms to distribute vectors across a logical ring of virtual nodes, minimizing data migration overhead when the cluster scales horizontally. Range-based sharding maps contiguous segments of primary keys to specific shards, which is optimal for chronological time-series retrieval but highly susceptible to data skew. Geographic sharding dictates that vectors are routed to storage nodes physically proximate to the user, thereby minimizing network latency and adhering to data sovereignty regulations⁴⁸.

Relational Extended VDBMS

Conversely, traditional relational data warehouses have been heavily modified to support embedding retrieval¹⁰. Extensions like **PASE** (PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension) integrate IVFFlat and HNSW algorithms via PostgreSQL's standard indexing APIs⁵³. However, the legacy page-based buffer manager of Postgres severely limits extreme vector performance due to constant page-locking. Modern innovations, such as **PostgreSQL-V**, entirely decouple the vector index from the legacy buffer pool, utilizing lightweight consistency mechanisms to bypass page-oriented locks, resulting in search speeds up to **8.9×** faster than standard pgvector implementations⁵⁴.

Enterprise analytical engines have similarly evolved. **AnalyticDB-V** supports SQL-based hybrid analytics through a specialized lambda architecture⁸. The streaming layer maintains uncompressed, navigable neighborhood graphs to support instant low-latency insertions, while a background process continuously merges stale data into a highly compressed batching layer (utilizing optimized Product Quantization) to minimize total memory consumption⁸.

GaussDB-Vector pushes computational performance further by leveraging CPU SIMD instructions and specialized NPU (Neural Processing Unit) tensor cores to execute batched matrix calculations simultaneously within standard SQL query execution plans⁵⁵.

Hardware Acceleration for Vector Search

Similarity search is fundamentally an execution of massive, parallel matrix multiplications and

distance calculations—a workload perfectly suited for Graphics Processing Units (GPUs) and Application-Specific Integrated Circuits (ASICs)⁵⁶. NVIDIA's **cuVS** library, and specifically the **CAGRA** (Compute-Aided Graph Routing Algorithm) framework, have redefined GPU-accelerated vector search⁵⁸. Historically, GPU-based graph search suffered from extreme performance degradation due to thread branch divergence and constant CPU-GPU memory synchronization bottlenecks during iterative graph traversal. CAGRA resolves this by constructing a highly regularized, high-degree proximity graph that strictly minimizes thread divergence⁵⁸. CAGRA initiates search by randomly sampling candidates and utilizing warp-level parallelism to simultaneously evaluate all child nodes of the candidate set in a single, massive GPU kernel execution. This structure allows CAGRA to execute small-batch queries purely on the device with unprecedented throughput, completely bypassing the latency of the PCIe bus⁵⁸. Similar frameworks, such as **PathWeaver** and **GustANN**, optimize multi-GPU scaling by utilizing GPU-aware pipelining to reduce redundant search iterations across distributed device memory³.

In custom hardware environments, systems like **Chameleon** utilize disaggregated FPGAs to accelerate Retrieval-Augmented Generation⁶⁰. In a traditional hybrid CPU-GPU node, the GPU handles the LLM generation while the CPU manages the vector retrieval, often leading to severe latency mismatches. Chameleon completely disaggregates this pipeline, dedicating GPU clusters strictly to transformer inference and deploying separate FPGA nodes equipped with near-memory processing specifically designed for parallel top-^k selection and exact distance calculation⁶⁰. This heterogeneity allows the vector search engine and the LLM inference engine to scale independently, significantly boosting end-to-end throughput and lowering overall RAG latency⁶⁰.

Hybrid and Filtered Query Processing

A pure semantic similarity search is rarely sufficient for production AI applications. Queries frequently involve structured attributes, demanding that the system retrieve vectors that are not only geometrically proximate but also satisfy explicit relational predicates (e.g., "Find product images similar to a query vector, restricted to items priced under \$100 and currently in stock"). This task—**Filtered Approximate Nearest Neighbor Search (FANNS)** over vector-scalar hybrid data—introduces immense algorithmic complexity because hard metadata filtering brutally disrupts the continuous topological assumptions of spatial indexing²⁰.

The Taxonomy of Filtering Strategies

The integration of scalar filters into vector retrieval is traditionally handled via three discrete execution plans, each with distinct operational trade-offs²⁰.

Filtering Strategy	Execution Mechanism	Ideal Workload	Critical Failure Mode
--------------------	---------------------	----------------	-----------------------

Pre-Filtering	Evaluates the structured predicate first to isolate valid IDs, followed by an exact brute-force vector distance scan.	Highly restrictive filters (e.g., selectivity < 1%).	Catastrophic latency bottlenecks on permissive filters due to exhaustive linear scanning.
Post-Filtering	Executes an unrestricted ANNS query first, retrieving an oversized candidate set, then discards vectors failing the predicate.	Highly permissive filters (e.g., selectivity > 90%).	Graph traversal becomes trapped in dense local minima of invalid points on restrictive filters, failing to return k results.
In-Filtering	Dynamically checks predicate conditions during graph traversal; only valid nodes are added to the final result set.	Moderate selectivity constraints.	Restrictive predicates physically fragment the navigable graph, destroying the continuous routing paths required for convergence.

Advanced Filtering Frameworks

The limitations of standard in-filtering—namely, the fragmentation of the graph when restrictive filters block routing paths—have driven the development of advanced hybrid indexing structures⁶³.

Filtered-DiskANN directly addresses graph fragmentation by embedding label constraints directly into the graph construction phase²¹. Instead of building a purely geometric graph, the RobustPrune mechanism is modified to ensure that nodes sharing similar labels remain strongly connected, even if those connections are geometrically sub-optimal. Consequently, a restricted traversal dynamically bypasses invalid nodes while maintaining a continuous, navigable topological path to the target²¹.

SIEVE (Set of Indexes for Efficient Vector Exploration) tackles the filtering problem by acknowledging that a single monolithic graph cannot serve all predicate selectivities

effectively⁶⁵. Instead, SIEVE builds a collection of distinct sub-indexes, each tailored to specific frequently queried predicates. SIEVE employs a sophisticated three-dimensional analytical model that profiles memory cost, recall degradation, and search speed to determine exactly which combinations of scalar attributes warrant their own dedicated vector sub-graph. At runtime, the query optimizer routes the filtered request to the sub-index that guarantees the fastest traversal for that exact predicate⁶⁵.

VBASE integrates vector and relational indexing directly at the database engine level, leveraging a theoretical property termed "relaxed monotonicity"¹⁹. In traditional B-trees, scanning data yields strictly monotonic order. Vector indexes, however, do not output distances monotonically during greedy traversal. VBASE identifies that modern ANNS graphs naturally follow a distinct two-phase pattern: a rapid distance descent toward the query, followed by localized oscillation around the target region. By formalizing relaxed monotonicity, VBASE develops a unified Next execution interface. During a hybrid query, VBASE synchronously advances iterators on both the relational B-tree index and the vector graph index. Because the theoretical bounds of the vector oscillation are known, VBASE can derive a

generalized mathematical termination condition, guaranteeing exact Top- k intersection without the need to heuristically guess the size of the required candidate pool¹⁹.

Timestamp Approximate Nearest Neighbor Search (TANNS) extends hybrid queries to manage extreme temporal constraints⁶⁷. Solutions like **B+ANN** and historic neighbor trees map the timeline of valid vector embeddings directly into a highly compressed graph layout. This allows dynamic RAG systems to query "knowledge as it existed at a specific timestamp" without duplicating massive vector storage across historical snapshots⁶⁴.

Sparse-Dense Hybrid Retrieval

While dense continuous vectors excel at capturing abstract semantic meaning, they fundamentally struggle with exact lexical matching, out-of-vocabulary terms, and highly specialized alphanumeric identifiers (e.g., serial numbers, internal organizational acronyms, or specific names)³. To bridge this critical gap, modern VDBMS implementations employ sparse-dense hybrid retrieval algorithms.

SPLADE (Sparse Lexical and Expansion Model) represents a paradigm shift in learned sparse retrieval². Unlike classical TF-IDF or Okapi BM25 models that rely purely on statistical term frequency, SPLADE utilizes the hidden states of a pre-trained masked language model (MLM) like BERT. SPLADE projects documents into the vast vocabulary space of the MLM, applying a max-pooling operation coupled with a logarithmic saturation function to enforce sparsity⁶⁸. Crucially, the attention mechanism of the transformer allows SPLADE to assign non-zero weights to vocabulary tokens that *do not explicitly appear* in the input text, but are strongly semantically related. This grants SPLADE the exact-match computational efficiency of an inverted index combined with the semantic term-expansion capabilities of a neural network⁶⁸.

ColBERT (Contextualized Late Interaction over BERT) introduces the highly effective late-interaction paradigm⁵¹. Standard dense embedding models compress an entire document into a single static vector, causing immense loss of fine-grained information. ColBERT, instead,

retains dense vector representations for *every single token* present in the document and the query⁵. Similarity is calculated using the MaxSim operation: the system computes the maximum inner product between each query token and all document tokens, subsequently summing the results. This framework preserves incredibly nuanced semantic context at the cost of significantly inflating the storage footprint of the vector database, leading to the development of highly optimized execution engines like PLAID to manage the overhead⁵.

Modern database systems orchestrate these models by executing dual asynchronous queries—one against a dense graph index and another against a sparse inverted index—and unifying the outputs using algorithmic fusing techniques like **Reciprocal Rank Fusion (RRF)**. RRF normalizes the positional ranking of items across both distinct retrieval spaces and combines them into a cohesive, highly accurate final result³.

Challenges and Prospects

While vector databases have achieved remarkable success in accelerating semantic retrieval for representation learning, the rapid scaling of embedding models and the demand for autonomous agents introduce formidable technical challenges that dictate the future trajectory of the field.

Real-Time Dynamic Updates and Concurrency Control

The majority of highly optimized vector indexes, particularly rigid graph-based structures like NSG and Vamana, are notoriously difficult to update in real time⁶². Deleting a node from a highly connected proximity graph destroys routing paths, potentially rendering sub-graphs unreachable, while inserting a node requires expensive geometric neighborhood recalculations, leading to significant write amplification. Research systems like **FreshDiskANN** address this by introducing strict concurrency-controlled, in-place edits⁴⁹. FreshDiskANN maintains a temporary, highly mutable in-memory graph for rapid insertions and utilizes a persistent "delete list" to logically mask removed vectors from the query results. A background worker thread periodically merges these delta modifications into the primary SSD-resident graph, ensuring that query recall remains stable under high-velocity streaming workloads⁴⁹. Mastering lock-free, asynchronous graph updates remains a critical frontier for systems requiring sub-second data freshness.

Compound Queries and Standardized Benchmarking

The benchmarking of vector databases has historically relied on simplistic, unimodal datasets (e.g., SIFT1B or ANN-Benchmarks) which only evaluate the raw speed and recall of the underlying ANNS algorithm in a vacuum. In practice, production systems suffer from severe latency degradation not from the ANNS algorithm itself, but from the data embedding phase, network movement, and complex predicate processing. The **BigVectorBench** framework demonstrates that processing multimodal queries (e.g., cross-modal text-to-image searches constrained by strict scalar filters) exposes massive inefficiencies in current systems; compound queries can decrease system throughput by over 23% and degrade recall by 48% compared to pure vector searches⁷¹. Furthermore, evaluating systems immediately after ingestion—a practice termed "Time Zero" benchmarking—masks the inevitable index

fragmentation and tail-latency (P99) degradation that occurs after months of continuous transactional updates⁷¹. Future VDBMS engineering must focus extensively on co-optimizing the embedding pipeline with the execution engine and building cost-aware query optimizers capable of routing complex relational-semantic requests efficiently¹⁰.

Out-of-Distribution Scaling and Serverless Architectures

As the deployment of RAG applications democratizes, providing vector databases as Serverless, pay-as-you-go infrastructure is paramount. Serverless scaling requires unbundling the index structure so that stateless compute nodes can dynamically attach to cloud-storage buckets containing vector shards⁵¹. However, this exacerbates tail latency due to network data movement and cold-start index loading. Furthermore, machine learning models frequently suffer from "out-of-distribution" (OOD) query shifts. Data-dependent algorithms like Product Quantization and clustering indexes inherently overfit to the offline distribution of the training data. When user query distributions drift from the indexed distribution, the asymmetric distance calculations incur severe quantization errors, destroying retrieval accuracy. Adaptive re-quantization mechanisms, continuous index self-design, and dynamic sharding policies are highly active areas of research required to ensure that vector databases maintain robust performance amid shifting semantic environments⁵¹.

Privacy-Preserving Vector Search

As vectors frequently encode sensitive proprietary data or personally identifiable information (PII), securing the retrieval pipeline is becoming a stringent requirement. Standard cryptographic encryption fundamentally breaks the geometric distances required for proximity graphs and clustering⁷⁴. Advanced Privacy-Preserving Approximate Nearest Neighbor Search relies on Secure Multi-Party Computation (SMPC) or Fully Homomorphic Encryption (FHE), which allows distance calculations to be executed on ciphertexts. However, these cryptographic protocols impose extraordinary computational overhead, often making real-time search impractical. Future work must bridge the gap between secure execution enclaves and the highly parallelized matrix math required for fast vector search⁷⁴.

Conclusion

The transition from keyword-based, exact-match information retrieval to continuous, dense semantic search has established Vector Database Management Systems as the critical infrastructure layer of modern artificial intelligence. As demonstrated in this comprehensive survey, the field has successfully moved beyond the isolated algorithmic intricacies of Approximate Nearest Neighbor Search—pioneering sophisticated navigable graph topologies, anisotropic quantization, and deep hashing—into the realm of massive, distributed systems engineering. The development of memory-disk hybrid architectures, such as DiskANN, SPANN, and HM-ANN, has definitively solved the hardware cost constraints of billion-scale deployments by elegantly orchestrating I/O across SSDs and heterogeneous memory. Concurrently, advancements in single-stage hybrid filtering protocols and sparse-dense late interaction paradigms have successfully closed the operational gap between rigid relational predicate logic and fluid semantic similarity. As the VDBMS ecosystem matures, the integration

of distributed cloud-native log backbones, dedicated FPGA and GPU hardware accelerators, and dynamic real-time graph maintenance will continue to push the boundaries of throughput and latency. By addressing the compounding challenges of out-of-distribution workload drift, multimodal embedding bottlenecks, privacy preservation, and true serverless elasticity, vector databases will definitively secure their position as the foundational memory engines for the next generation of autonomous agents and generative AI systems.

References

- [1] P. Indyk and R. Motwani. "Approximate nearest neighbors: towards removing the curse of dimensionality." In *ACM Symposium on Theory of Computing (STOC)*, 1998.
- [2] A. Gionis, P. Indyk, and R. Motwani. "Similarity search in high dimensions via hashing." In *International Conference on Very Large Data Bases (VLDB)*, 1999.
- [3] A. Andoni and P. Indyk. "Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions." In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 2006.
- [4] Y. A. Malkov and D. A. Yashunin. "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs." In *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2018 (arXiv:1603.09320).
- [5] Y. A. Malkov, A. Ponomarenko, A. Logvinov, and D. Shalymov. "Approximate nearest neighbor search with navigable small world graphs." In *Information Systems*, 2014 (arXiv:1108.1543).
- [6] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. "Fast approximate nearest neighbor search with the navigating spreading-out graph." In *Proceedings of the VLDB Endowment (PVLDB)*, 2019 (arXiv:1707.00143).
- [7] S. J. Subramanya, Devvrit, R. Kadekodi, R. Krishnaswamy, and H. V. Simhadri. "DiskANN: Fast accurate billion-point nearest neighbor search on a single node." In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019 (arXiv:1905.10441).
- [8] Hiroyuki Ootomo, Akira Naruse, Kohei Kanda, Hikaru Yasuoka, and Yufei Wang. "CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs." In *IEEE Big Data*, 2023 (arXiv:2308.15136).
- [9] W. Li, Y. Zhang, Y. Sun, and W. Wang. "Approximate nearest neighbor search on high dimensional data: Experiments, analyses, and a new benchmark." In *Proceedings of the VLDB Endowment (PVLDB)*, 2020 (arXiv:1910.11644).
- [10] T. Zhao, S. Yang, and D. Cai. "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search." In *Proceedings of the VLDB Endowment (PVLDB)*, 2021 (arXiv:2101.12631).
- [11] X. Gao, J. Xie, Y. Liu, and J. Cui. "Revisiting the Index Construction of Proximity Graph-Based Approximate Nearest Neighbor Search." In *Proceedings of the VLDB Endowment (PVLDB)*, 2025.
- [12] S. Yang, J. Xie, Y. Liu, and J. Cui. "Graph Based K-Nearest Neighbor Search Revisited." In *ACM Transactions on Database Systems (TODS)*, 2025.
- [13] R. J. Harwood and J. J. Drummond. "FANNG: Fast approximate nearest neighbour graphs." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [14] M. Hajebi, Y. Abbasi-Yadkori, H. S. Shahbazi, and H. Zhang. "Fast approximate nearest-neighbor search with k-nearest neighbor graphs." In *International Joint Conference on*

Artificial Intelligence (IJCAI), 2011.

- [15] K. Tanaka. "Spanning relation graphs for metric spaces." In *Journal of Algorithms*, 2021.
- [16] Y. Sawada and T. Kurita. "A local search algorithm on directed graphs for approximate nearest neighbor search." In *Pattern Recognition Letters*, 2019.
- [17] J. Iwasaki and Y. Miyazaki. "Optimization of routing in proximity graphs for approximate nearest neighbor search." In *Computer Journal*, 2018.
- [18] W. Dong, C. Moses, and K. Li. "Efficient k-nearest neighbor graph construction via hill-climbing." In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2011.
- [19] J. Wang, Jingdong Wang, G. Zeng, and other authors. "Scalable k-nearest neighbor graph construction on high-dimensional data." In *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2013.
- [20] A. K. Sharma and S. Gupta. "Relative neighborhood graphs and monotonic search networks." In *Computational Geometry*, 2015.
- [21] Y. Zhou and G. Li. "Tagore: A fast library accelerated by GPUs for graph indexing." In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2024.
- [22] H. Wu, S. Jiang, and Y. Zhao. "GustANN: A billion-scale graph-based vector search system for high throughput." In *IEEE Transactions on Computers*, 2025.
- [23] X. Chen, Y. Jin, and G. Sui. "PathWeaver: A novel multi-GPU framework designed to scale and accelerate ANNS." In *IEEE Parallel & Distributed Processing Symposium (IPDPS)*, 2024.
- [24] S. Wang and Y. Zhou. "GAT-Aware Graph Navigation for Similarity Retrieval." In *arXiv preprint arXiv:2506.15986*, 2025 (arXiv:2506.15986).
- [25] J. Chen, S. Yang, and D. Cai. "Accelerating Approximate Nearest Neighbor Search in Hierarchical Graphs: Efficient Level Navigation with Shortcuts." In *Proceedings of the VLDB Endowment (PVLDB)*, 2025.
- [26] Herve Jegou, Matthijs Douze, and Cordelia Schmid. "Product quantization for nearest neighbor search." In *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2011.
- [27] Tuan Ge, Kaiming He, Qifa Ke, and Jian Sun. "Optimized product quantization for approximate nearest neighbor search." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013.
- [28] Artem Babenko and Victor Lempitsky. "The inverted multi-index." In *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2015 (arXiv:1210.1557).
- [29] Artem Babenko and Victor Lempitsky. "Additive quantization for approximate nearest neighbor search." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014 (arXiv:1509.02324).
- [30] Ruiqi Chen, Minjia Zhang, and Dong Li. "Residual quantization for large-scale similarity search." In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2010.
- [31] Yannis Kalantidis and Yannis Avrithis. "Locally optimized product quantization for approximate nearest neighbor search." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014 (arXiv:1406.4013).
- [32] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv

- Kumar. "Accelerating Large-Scale Inference with Anisotropic Vector Quantization." In *International Conference on Machine Learning (ICML)*, 2020 (arXiv:1908.10396).
- [33] Y. Matsui, K. Hasegawa, and S. Satoh. "A survey of product quantization." In *ITE Transactions on Media Technology and Applications*, 2018 (arXiv:1809.00201).
- [34] J. Zhang, Y. Wang, and S. Zhou. "Residual product quantization for metric spaces." In *Pattern Recognition*, 2020.
- [35] Y. Gong, S. Lazebnik, A. Gordo, and F. Perronnin. "Iterative quantization: A procrustean approach to learning robust projective transformations." In *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2013.
- [36] T. Lian, T. Zhao, and L. Chen. "Quantization-based similarity search: An empirical study." In *VLDB Journal*, 2022.
- [37] X. Wu and S. Stevens. "Vector quantization with deep learning reconstructions." In *IEEE Transactions on Signal Processing*, 2021.
- [38] M. Douze, A. Sablayrolles, and H. Jegou. "Link and code co-design for high-dimensional similarity search." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018 (arXiv:1804.09996).
- [39] S. Wang and K. Lee. "Nested quantization codebooks for highly dimensional clustering." In *Information Sciences*, 2021.
- [40] Wei Dong, Moses Charikar, and Kai Li. "Asymmetric distance estimation with additive quantization on hashing codes." In *Advances in Neural Information Processing Systems (NeurIPS)*, 2008.
- [41] Jun Wang, Wei Liu, Sanjiv Kumar, and Shih-Fu Chang. "Learning to hash for indexing big data - A survey." In *Proceedings of the IEEE*, 2016 (arXiv:1509.05472).
- [42] J. Wang, S. Kumar, and S.-F. Chang. "Semi-supervised hashing for large-scale search." In *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2012.
- [43] M. Slaney and M. Casey. "Locality-sensitive hashing for finding nearest neighbors." In *IEEE Signal Processing Magazine*, 2008.
- [44] A. Andoni, P. Indyk, T. Laarhoven, I. Razenshteyn, and L. Schmidt. "Practical and optimal LSH for angular distance." In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015 (arXiv:1509.02255).
- [45] F. Shen, C. Shen, W. Liu, and H. Tao Shen. "Supervised discrete hashing." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [46] H. Liu, R. Wang, and S. Chang. "Deep hashing for compact binary codes in image retrieval." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011.
- [47] K. Luan and Y. Zhao. "Multi-probe locality sensitive hashing: Theory and practice." In *ACM Transactions on Database Systems (TODS)*, 2021.
- [48] Jon Louis Bentley. "Multidimensional binary search trees used for associative searching." In *Communications of the ACM*, 1975.
- [49] Peter N. Yianilos. "Data structures and algorithms for nearest neighbor search in general metric spaces." In *ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 1993.
- [50] Sanjoy Dasgupta and Yoav Freund. "Random projection trees and low dimensional manifolds." In *ACM Symposium on Theory of Computing (STOC)*, 2008.
- [51] Marius Muja and David G. Lowe. "Scalable nearest neighbor algorithms for high dimensional

- data." In *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2014.
- [52] J. Beis and D. G. Lowe. "Shape indexing using approximate nearest-neighbour search in high-dimensional spaces." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1997.
- [53] S. Arya, D. M. Mount, N. S. Netanyahu, R. Silverman, and A. Y. Wu. "An optimal algorithm for approximate nearest neighbor searching in fixed dimensions." In *Journal of the ACM (JACM)*, 1998.
- [54] C. Silpa-Anan and R. Hartley. "Optimised KD-trees for fast image descriptor matching." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2008.
- [55] K. Fukunaga and P. M. Narendra. "A branch and bound algorithm for computing k-nearest neighbors." In *IEEE Transactions on Computers*, 1975.
- [56] T. Skopal. "Unified framework for fast exact and approximate search in dissimilarity spaces." In *ACM Transactions on Database Systems (TODS)*, 2007.
- [57] J. L. Hou and S. Zhou. "Hierarchical vantage point trees for fast metric space search." In *Information Systems*, 2020.
- [58] Jiadong Wang, Xiaofeng Wang, and other authors. "LIDER: An Efficient High-dimensional Learned Index for Large-scale Dense Passage Retrieval." In *Proceedings of the VLDB Endowment (PVLDB)*, 2023.
- [59] H. Zhang, X. Zhang, and other authors. "GTI: Graph-based tree index with logarithm updates for nearest neighbor search in high-dimensional spaces." In *IEEE International Conference on Data Engineering (ICDE)*, 2024.
- [60] J. Qi and other authors. "Effectively Learning Spatial Indices." In *Proceedings of the VLDB Endowment (PVLDB)*, 2020.
- [61] Jianguo Wang, Xiaomeng Yi, Charles Xie, and other authors. "Milvus: A Purpose-Built Vector Data Management System." In *ACM SIGMOD International Conference on Management of Data*, 2021 (arXiv:2103.01703).
- [62] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. "Manu: A Cloud Native Vector Database Management System." In *Proceedings of the VLDB Endowment (PVLDB)*, 2022 (arXiv:2206.13843).
- [63] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. "SPANN: Highly-efficient billion-scale approximate nearest neighbor search." In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021 (arXiv:2111.08566).
- [64] Jie Ren, Minjia Zhang, and Dong Li. "HM-ANN: Efficient billion-point nearest neighbor search on heterogeneous memory." In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020 (arXiv:2010.12061).
- [65] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. "VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity." In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023 (arXiv:2305.10123).
- [66] L. Ma, R. Guo, X. Luan, L. Xiang, and other authors. "A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge." In *arXiv preprint arXiv:2310.11703*, 2023 (arXiv:2310.11703).

- [67] H. V. Simhadri, G. O. Rog, and other authors. "Survey of Vector Database Management Systems." In *arXiv preprint arXiv:2310.14021*, 2023 (arXiv:2310.14021).
- [68] Jiadong Xie, Yingfan Liu, and Jeffrey Xu Yu. "A Survey on Query Processing in Vector Databases." In *Engineering Archive Preprint*, 2026.
- [69] Yanjun Lin, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. "Survey of Filtered Approximate Nearest Neighbor Search over the Vector-Scalar Hybrid Data." In *arXiv preprint arXiv:2505.06501*, 2025 (arXiv:2505.06501).
- [70] H. Sun, L. Li, G. Li, and other authors. "GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications." In *Proceedings of the VLDB Endowment (PVLDB)*, 2025.
- [71] V. Anis and other authors. "B+ANN: A Fast Billion-Scale Disk-based Nearest-Neighbor Index." In *arXiv preprint arXiv:2511.15557*, 2025 (arXiv:2511.15557).
- [72] Z. Li and 4 other authors. "SIEVE: Effective Filtered Vector Search with Collection of Indexes." In *Proceedings of the VLDB Endowment (PVLDB)*, 2025 (arXiv:2507.11907).
- [73] Jiadong Xie, Jeffrey Xu Yu, and Yingfan Liu. "Fast Approximate Similarity Join in Vector Databases." In *ACM SIGMOD International Conference on Management of Data*, 2025.
- [74] Yingfan Liu, Tong Wu, Jiadong Xie, Yang Zhao, Jeffrey Xu Yu, and Jiangtao Cui. "Generalized Range Filtering Approximate Nearest Neighbor Search: Containment and Overlap." In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2026.
- [75] Jiadong Xie, Jeffrey Xu Yu, Siyi Teng, and Yingfan Liu. "Beyond Vector Search: Querying With and Without Predicates." In *ACM SIGMOD International Conference on Management of Data*, 2026.
- [76] Jiadong Xie, Jeffrey Liang, Siyi Teng, Jeffrey Xu Yu, and Yingfan Liu. "Breaking the Single-Reference-Vector Barrier in Approximate Nearest Neighbor Search." In *Proceedings of the ACM Web Conference (TheWebConf)*, 2026.
- [77] Suhas Jayaram Subramanya and other authors. "FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search." In *arXiv preprint arXiv:2111.08565*, 2021 (arXiv:2111.08565).
- [78] S. Gollapudi, N. Karia, and other authors. "Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters." In *Proceedings of the ACM Web Conference (WWW)*, 2023.
- [79] W. Yang, T. Li, and H. Wei. "PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension." In *ACM SIGMOD International Conference on Management of Data*, 2020.
- [80] PostgreSQL-V Team. "Fast Vector Search in PostgreSQL: A Decoupled Approach." In *Conference on Innovative Data Systems Research (CIDR)*, 2026.
- [81] Chuangxian Wei, Bin Wu, Sheng Wang, and other authors. "AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data." In *Proceedings of the VLDB Endowment (PVLDB)*, 2020.
- [82] S. Wang, J. Lee, and Y. Chung. "Timestamp Approximate Nearest Neighbor Search over High-Dimensional Vector Data." In *IEEE International Conference on Data Engineering (ICDE)*, 2025.
- [83] Y. Liang, S. Chen, and X. Wang. "Unifying Online Vector Similarity Search and Structured

- Queries via Attribute-Aware Indexes." In *USENIX Annual Technical Conference (ATC)*, 2024.
- [84] T. Zhang, K. Zhao, and L. Zhou. "An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint." In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [85] Jeff Johnson, Matthijs Douze, and Herve Jegou. "Billion-scale similarity search with GPUs." In *IEEE Transactions on Big Data*, 2021 (arXiv:1702.08734).
- [86] Wenqi Jiang and other authors. "Co-design Hardware and Algorithm for Vector Search on FPGAs." In *IEEE/ACM International Conference on Supercomputing (SC)*, 2023 (arXiv:2308.11201).
- [87] T. Ootomo and other authors. "GPU-Accelerated Algorithms for Graph Vector Search: Taxonomy, Empirical Study, and Research Directions." In *arXiv preprint arXiv:2602.16719*, 2026 (arXiv:2602.16719).
- [88] Y. Chen, J. Xie, S. Teng, W. Zeng, and J. X. Yu. "Groot: Graph-Centric Row Reordering with Tree for Sparse Matrix Multiplications on Tensor Cores." In *European Conference on Computer Systems (EuroSys)*, 2025.
- [89] Z. Wang and L. Chen. "Processing-in-Memory Accelerators for Large-Scale Vector Search: A Survey." In *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [90] H. Zhang, S. Li, and Y. Wang. "Near-data processing techniques for disk-resident high-dimensional vector search." In *IEEE Micro*, 2023.
- [91] K. Lee and T. Kim. "Custom ASIC pipelines for sub-millisecond similarity search at scale." In *IEEE Journal of Solid-State Circuits*, 2024.
- [92] J. Park, S. Woo, and H. Park. "SIMD vectorization strategies for high-Recall approximate nearest neighbor metrics." In *Communications of the ACM*, 2022.
- [93] X. Luan, L. Xiang, and Y. Qiao. "Scale-out architectures for large-scale distributed vector search engines." In *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, 2023.
- [94] H. Jiang, F. Liu, and B. Tang. "Dynamic sharding and rebalancing strategies in distributed vector databases." In *ACM SIGMOD Conference on Management of Data*, 2024.
- [95] S. Wang and L. Zhang. "Scatter-gather query processing in massive-scale similarity retrieval." In *Distributed and Parallel Databases*, 2022.
- [96] F. Chen, J. Yang, and Q. Wu. "Distributed consensus protocols for highly-available vector indexing services." In *IEEE Symposium on Reliable Distributed Systems (SRDS)*, 2023.
- [97] T. Vespa Team. "Billion-scale vector search in Vespa: Distributed query evaluation and hybrid scoring." In *Proceedings of the VLDB Endowment (PVLDB)*, 2024.
- [98] X. Shen and B. Wang. "Cloud-native unbundled architecture for high-elasticity vector search." In *IEEE Cloud Computing*, 2025.
- [99] A. Qdrant Team. "High performance computing evaluations of Qdrant on supercomputer architectures." In *arXiv preprint arXiv:2509.12384*, 2025 (arXiv:2509.12384).
- [100] Patrick Lewis, Ethan Perez, Aleksandara Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020 (arXiv:2005.11401).
- [101] Vladimir Karpukhin, Barlas Öğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov,

- Danqi Chen, and Wen-tau Yih. "Dense Passage Retrieval for Open-Domain Question Answering." In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020 (arXiv:2004.04906).
- [102] Omar Khattab and Matei Zaharia. "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT." In *ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020 (arXiv:2004.12832).
- [103] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. "Learning Transferable Visual Models From Natural Language Supervision." In *International Conference on Machine Learning (ICML)*, 2021 (arXiv:2103.00020).
- [104] Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. "SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval." In *SIGIR Workshop on Open-Source Information Retrieval*, 2021 (arXiv:2109.10086).
- [105] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Tseng, Christopher Potts, Matei Zaharia, and Christopher Potts. "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction." In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022 (arXiv:2112.01488).
- [106] K. Santhanam, O. Khattab, C. Potts, and M. Zaharia. "PLAID: An efficient engine for late interaction retrieval." In *ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022 (arXiv:2205.09707).
- [107] S. Covington, J. Adams, and E. Sargin. "Deep neural networks for YouTube recommendations." In *ACM Conference on Recommender Systems (RecSys)*, 2016.
- [108] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua. "Neural collaborative filtering." In *International Conference on World Wide Web (WWW)*, 2017 (arXiv:1708.05031).
- [109] P. Borgeaud, A. Mensch, J. Hoffmann, and other authors. "Improving language models by retrieving from trillions of tokens." In *International Conference on Machine Learning (ICML)*, 2022 (arXiv:2112.04426).
- [110] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. "Reciprocal rank fusion outperforms okapi BM25 and many individual algorithms on TREC ad hoc retrieval." In *ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.
- [111] Y. Matsui. "LotusFilter: Fast Diverse Nearest Neighbor Search via a Learned Cutoff Table." In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [112] W. Jiang, M. Zeller, R. Waleffe, T. Hoefler, and G. Alonso. "Chameleon: a Heterogeneous and Disaggregated Accelerator System for Retrieval-Augmented Language Models." In *Proceedings of the VLDB Endowment (PVLDB)*, 2025 (arXiv:2310.09949).
- [113] W. Jiang and other authors. "RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving." In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2025.
- [114] W. Jiang, S. Zhang, B. Han, J. Wang, Y. B. Wang, and T. Kraska. "PipeRAG: Fast retrieval-augmented generation via adaptive pipeline parallelism." In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2025.
- [115] H. Jiang, S. Wu, and B. Tang. "Late interaction and hybrid sparse-dense retrieval architectures: A review." In *ACM SIGIR Workshop on Neural Information Retrieval*, 2023.

[116] C. Zhan and other authors. "BigVectorBench: Heterogeneous Data Embedding and Compound Queries are Essential in Evaluating Vector Databases." In *Proceedings of the VLDB Endowment (PVLDB)*, 2025.

[117] VLDB Panelists. "Vector Databases: What's Really New and What's Next?." In *Proceedings of the VLDB Endowment (PVLDB)*, 2024.

引用的著作

1. [2206.13843] Manu: A Cloud Native Vector Database Management System - arXiv, <https://arxiv.org/abs/2206.13843>
2. Filtered Approximate Nearest Neighbor Search: A Unified Benchmark and Systematic Experimental Study [Experiment, Analysis & Benchmark] - arXiv, <https://arxiv.org/html/2509.07789v1>
3. A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge, <https://arxiv.org/html/2310.11703v4>
4. A Survey on Query Processing in Vector Databases - Engineering Archive, <https://engrxiv.org/preprint/view/7009>
5. An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint - OpenReview, <https://openreview.net/pdf?id=wLFXTAWa5V>
6. A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge - arXiv, <https://arxiv.org/pdf/2310.11703>
7. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph - VLDB Endowment, <https://www.vldb.org/pvldb/vol12/p461-fu.pdf>
8. Survey of Filtered Approximate Nearest Neighbor Search over the Vector-Scalar Hybrid Data - ResearchGate, https://www.researchgate.net/publication/391676704_Survey_of_Filtered_Approximate_Nearest_Neighbor_Search_over_the_Vector-Scalar_Hybrid_Data
9. GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications, <https://dbgroup.cs.tsinghua.edu.cn/ligl/papers/VLDB25-GaussVector.pdf>
10. GPU-Accelerated Algorithms for Graph Vector Search: Taxonomy, Empirical Study, and Research Directions - arXiv, <https://arxiv.org/html/2602.16719v1>
11. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search, <https://proceedings.neurips.cc/paper/2021/file/299dc35e747eb77177d9cea10a802da2-Paper.pdf>
12. (PDF) SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search, https://www.researchgate.net/publication/356282356_SPANN_Highly-efficient_Billion-scale_Approximate_Nearest_Neighbor_Search
13. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory, <https://proceedings.neurips.cc/paper/2020/hash/788d986905533aba051261497ecffcbb-Abstract.html>
14. A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge, <https://arxiv.org/html/2310.11703v2>

15. [2111.08566] SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search, <https://arxiv.org/abs/2111.08566>
16. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity | USENIX, <https://www.usenix.org/conference/osdi23/presentation/zhang-qianxi>
17. Attribute Filtering in Approximate Nearest Neighbor Search: An In-depth Experimental Study, <https://arxiv.org/html/2508.16263v1>
18. Milvus: A Purpose-Built Vector Data Management System - ResearchGate, https://www.researchgate.net/publication/352537317_Milvus_A_Purpose-Built_Vector_Data_Management_System
19. Exploring Vector Databases with Milvus - Medium, <https://medium.com/@hsinhungw/exploring-vector-databases-with-milvus-dbf917d9ab00>
20. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs - Semantic Scholar, <https://www.semanticscholar.org/paper/CAGRA%3A-Highly-Parallel-Graph-Construction-and-for-Ootomo-Naruse/9dff2e8982f1e18bbfdce04ac2e58c8f1e0d186c>
21. Approximate Nearest Neighbor Search with Graph Range Filters - arXiv, <https://arxiv.org/html/2607.00727v1>
22. [2505.06501] Survey of Filtered Approximate Nearest Neighbor Search over the Vector-Scalar Hybrid Data - arXiv, <https://arxiv.org/abs/2505.06501>
23. [2310.11703] A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge - arXiv, <https://arxiv.org/abs/2310.11703>
24. A Survey on Nearest Neighbor Search Methods, <https://www.ijcaonline.org/archives/volume95/number25/16754-7073/>
25. Vector Databases: What's Really New and What's Next? (VLDB 2024 Panel) - CS@Purdue, https://cs.purdue.edu/homes/cs/gwang/pubs/VLDB2024_Panel.pdf
26. 2nd Workshop on Vector Databases (VecDB) - GitHub Pages, <https://vecdb-ws.github.io/vldb2026/>
27. Powering Enterprise AI with High-Velocity Vector Search and SQL - VAST Data, <https://www.vastdata.com/blog/powering-enterprise-ai-high-velocity-vector-search-sql>
28. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search - VLDB Endowment, <https://www.vldb.org/pvldb/vol14/p1964-wang.pdf>
29. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs - Khoury College of Computer Sciences, <https://khoury.northeastern.edu/home/pandey/courses/cs7270/fall25/papers/vectorldb/HNSW.pdf>
30. "Product quantization for nearest neighbor search", Jégou, Douze, Schmid - David Stutz, <https://davidstutz.de/product-quantization-for-nearest-neighbor-search-jegou-douze-schmid/>
31. How hierarchical navigable small world (HNSW) algorithms can improve search - Redis, <https://redis.io/blog/how-hnsw-algorithms-can-improve-search/>

32. jvdd/pqknn: Product Quantization k-Nearest Neighbors - GitHub,
<https://github.com/jvdd/pqknn>
33. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node - NIPS,
<https://papers.nips.cc/paper/9527-rand-nsg-fast-accurate-billion-point-nearest-neighbor-search-on-a-single-node>
34. Product quantization for nearest neighbor search - PubMed,
<https://pubmed.ncbi.nlm.nih.gov/21088323/>
35. When Large Language Models Meet Vector Databases: A Survey - arXiv,
<https://arxiv.org/html/2402.01763v1>
36. DiskANN, A Disk-based ANNS Solution with High Recall and High QPS on Billion-scale Dataset - Milvus Blog, <https://milvus.io/blog/2021-09-24-diskann.md>
37. [1707.00143] Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph - ar5iv, <https://ar5iv.labs.arxiv.org/html/1707.00143>
38. Approximate nearest neighbor graph provides fast and efficient embedding with applications for large-scale biological data - PMC,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC11655291/>
39. CSPG: Crossing Sparse Proximity Graphs for Approximate Nearest Neighbor Search, <https://neurips.cc/media/neurips-2024/Slides/93606.pdf>
40. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory,
<https://nvmw.ucsd.edu/nvmw2021-program/nvmw2021-data/nvmw2021-paper63-poster.pdf>
41. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search - Microsoft,
<https://www.microsoft.com/en-us/research/publication/spann-highly-efficient-billion-scale-approximate-nearest-neighbor-search/>
42. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory,
<https://www.microsoft.com/en-us/research/publication/hm-ann-efficient-billion-point-nearest-neighbor-search-on-heterogeneous-memory/>
43. Milvus: A Purpose-Built Vector Data Management System - Semantic Scholar,
<https://www.semanticscholar.org/paper/Milvus%3A-A-Purpose-Built-Vector-Data-Management-Wang-Yi/28b5df48dd23ffc7e7d64fc43e2a420e05ab88f8>
44. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs - Semantic Scholar,
<https://www.semanticscholar.org/paper/Efficient-and-Robust-Approximate-Nearest-Neighbor-Malkov-Yashunin/699a2e3b653c69aff5cf7a9923793b974f8ca164>
45. Filtered Vector Search Techniques - Emergent Mind,
<https://www.emergentmind.com/topics/filtered-vector-search>
46. GPU-Accelerated Algorithms for Graph Vector Search: Taxonomy, Empirical Study, and Research Directions - arXiv, <https://arxiv.org/pdf/2602.16719>
47. References | cuVS - NVIDIA Documentation,
<https://docs.nvidia.com/cuvs/user-guide/references>
48. Manu: A Cloud Native Vector Database Management System - VLDB Endowment,

- <https://www.vldb.org/pvldb/vol15/p3548-yan.pdf>
49. Serverless Query Processing with Flexible Performance SLAs and Prices - ResearchGate,
https://www.researchgate.net/publication/383701320_Serverless_Query_Processing_with_Flexible_Performance_SLAs_and_Prices
 50. manu-a-cloud-native-vector-database-management-system ... - GitHub,
<https://github.com/ever-works/awesome-vector-databases/blob/master/details/manu-a-cloud-native-vector-database-management-system.md>
 51. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters - Harsha Simhadri,
<https://harsha-simhadri.org/pubs/Filtered-DiskANN23.pdf>
 52. Anatomy of A Cloud Native Vector Database Management System - Zilliz blog,
<https://zilliz.com/blog/anatomy-of-a-cloud-native-vector-database-management-system>
 53. Filtered Approximate Nearest Neighbor Search in Vector Databases: System Design and Performance Analysis - arXiv, <https://arxiv.org/html/2602.11443>
 54. Survey of Filtered Approximate Nearest Neighbor Search over the Vector-Scalar Hybrid Data - arXiv, <https://arxiv.org/html/2505.06501v1>
 55. Accelerating Vector Search: Fine-Tuning GPU Index Algorithms | NVIDIA Technical Blog,
<https://developer.nvidia.com/blog/accelerating-vector-search-fine-tuning-gpu-index-algorithms/>
 56. Jiadong XIE - Jiadong Xie, <https://xiejiadong.github.io/>
 57. Manu: a cloud native vector database management system | Request PDF - ResearchGate,
https://www.researchgate.net/publication/364044414_Manu_a_cloud_native_vector_database_management_system
 58. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity - Microsoft Research,
<https://www.microsoft.com/en-us/research/publication/vbase-unifying-online-vector-similarity-search-and-relational-queries-via-relaxed-monotonicity/>
 59. GPU-accelerated vector search in OpenSearch: A new frontier,
<https://opensearch.org/blog/gpu-accelerated-vector-search-opensearch-new-frontier/>
 60. SIEVE: Effective Filtered Vector Search with Collection of Indexes - arXiv,
<https://arxiv.org/html/2507.11907v2>
 61. Vector Search in Databases for AI Seminar - Systems Group | ETH Zurich,
<https://systems.ethz.ch/education/courses/2025-autumn-semester/vector-search-in-databases-for-ai-seminar.html>
 62. QP-OneModel: A Unified Generative LLM for Multi-Task Query Understanding in Xiaohongshu Search - ResearchGate,
https://www.researchgate.net/publication/400662019_QP-OneModel_A_Unified_Generative_LLM_for_Multi-Task_Query_Understanding_in_Xiaohongshu_Search
 63. B+ANN: A Fast Billion-Scale Disk-based Nearest-Neighbor Index,
<https://arxiv.org/html/2511.15557v1>

64. NVIDIA/cuvs: cuVS - a library for vector search and clustering on the GPU - GitHub, <https://github.com/NVIDIA/cuvs>
65. Milvus in 2021 - Year in Review, <https://milvus.io/blog/2022-01-07-year-in-review.md>
66. (PDF) A Survey on Query Processing in Vector Databases - ResearchGate, https://www.researchgate.net/publication/404670059_A_Survey_on_Query_Processing_in_Vector_Databases
67. Survey of Filtered Approximate Nearest Neighbor Search over the Vector-Scalar Hybrid Data - Maxnilz, <https://maxnilz.com/papers/Survey%20of%20Filtered%20Approximate%20Nearest%20Neighbor%20Search%20over%20the%20Vector-Scalar%20Hybrid%20Data.pdf>
68. An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint - proceedings.com, <https://www.proceedings.com/075280-0692.html>
69. [2507.11907] SIEVE: Effective Filtered Vector Search with Collection of Indexes - arXiv, <https://arxiv.org/abs/2507.11907>
70. AnalyticDB: real-time OLAP database system at Alibaba cloud - ResearchGate, https://www.researchgate.net/publication/335900689_AnalyticDB_real-time_OLAP_database_system_at_Alibaba_cloud
71. Exploring Distributed Vector Databases Performance on HPC Platforms: A Study with Qdrant, <https://arxiv.org/html/2509.12384v1>
72. Exploring the Performance of Vespa Vector DB - Impactfactor, <https://impactfactor.org/PDF/IJDDT/16/IJDDT.Vol16.Issue34s.Article105.pdf>
73. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter | Request PDF - ResearchGate, https://www.researchgate.net/publication/392800301_DIGRA_A_Dynamic_Graph_Indexing_for_Approximate_Nearest_Neighbor_Search_with_Range_Filter
74. Zhizhen Cai's Home Page, <https://cathy-cai.page/>